LABORATORY 2

Laravel Product CRUD

Student study guide and demonstration handbook
Updated October 7, 2026

This guide explains the Product CRUD application you built. Read it beside your code rather than memorizing the words. All application paths below start inside `app-crud/`, which is the Laravel project folder.

1. What is complete?

The functional scope described in Activity No. 2 is implemented: Create, Read, Update, and Delete using Laravel routes, a controller, Blade views, an Eloquent model, and a database. You demonstrated saving a product, listing it, editing its quantity, cancelling a delete, and confirming deletion. You also checked browser validation, server validation, and narrow-screen layouts.

The dark glass design is your chosen presentation layer. The shared navbar now has the reference's theme button, switching between dark and light colors. Its outline icons are inline SVG with a Lucide-like style, not the lucide-react package. This project does not use React.

Functional completion is different from submitting the activity. Confirm your instructor's required screenshots, file naming, database export, and upload format before submission; the supplied PDF does not establish a detailed submission checklist. The screenshot plan below is preparation, not a claim that your instructor requires those exact images.

2. Where do I find each part?

File	Responsibility	What to show if asked
`routes/web.php`	Match a browser URL and HTTP method to a controller method	The six product routes and their names
`app/Http/Controllers/ProductController.php`	Coordinate CRUD and validate input	`index`, `create`, `store`, `edit`, `update`, `destroy`
`app/Models/Product.php`	Represent products and use Eloquent database methods	`extends Model` and `$fillable`
`database/migrations/2026_10_07_015613_create_products_table.php`	Describe products table structure	`up`, `down`, and the columns
`resources/views/layouts/app.blade.php`	Shared HTML shell, navbar, and stylesheet link	`@yield` and `asset`
`resources/views/products/index.blade.php`	Display products, actions, and success feedback	`@forelse`, the Edit link, and the Delete form
`resources/views/products/create.blade.php`	Add Product form	Form action, input names, `@csrf`, `old`, errors
`resources/views/products/edit.blade.php`	Edit Product form	`@method('PUT')` and saved-value fallbacks
`public/css/style.css`	Dark glass styling, spacing, focus, and responsive rules	Glass panels, forms, actions, and media queries
`public/js/theme.js`	Simple light/dark appearance toggle	Body class, click listener, and localStorage
`.env`	Local environment/database configuration	Explain DB keys without exposing secrets

An earlier editor tab may still show `resources/views/index.blade.php`. The controller calls `products.index`, so the active file is inside `views/products`; an older copy outside that folder, if retained, is not used by these routes. The generated `resources/css/app.css` belongs to the default Tailwind/Vite pipeline. Our layout links `public/css/style.css` directly instead. `routes/console.php` defines terminal commands; it does not define the CRUD URLs. `app/Http/Controllers/Controller.php` is the shared parent class, not the file containing our product operations. Laravel's generated files and `vendor/` contain framework infrastructure; they are not all student-written CRUD code.

3. Explain MVC using this application

Model: `Product.php` represents product records. Eloquent lets the controller request database operations using PHP objects and methods. The model itself is not the database table; it is the class used to interact with that table.

View: Blade templates present the data and collect user input. The list loops over products passed by the controller. The forms send input back through routes. Blade is rendered on the server; the browser receives HTML, not the original Blade directives.

Controller: `ProductController.php` coordinates the request. It reads or validates input, calls the model, and returns a view or redirect. It avoids mixing page markup and table-creation code into the request methods.

Routes: `web.php` connects a request to the correct controller method. Routes support MVC, but are not an extra letter in MVC.

Migration: A migration describes database structure. It is different from a model, which interacts with the records in that structure.

A practical sequence is:

```
Browser request -> Route -> Controller -> Model/Eloquent -> Database
                                  |
                          View receives the result
                                  |
                          Browser displays the HTML
```

The database returns data through Eloquent to the controller. Not every request needs a database operation: `create()` simply returns the blank form.

4. Follow one product through all four operations

Create: save Apple

1. The Add Product link uses `route('product.create')` to generate the form URL.
2. `GET /product/create` calls `create()`, returning `products.create`.
3. The form has `method="POST"` and targets `product.store`.
4. The input `name` attributes become submitted keys: `name`, `qty`, `price`, and `description`. HTML `id` attributes link fields to their labels.
5. `@csrf` supplies the token checked by Laravel's web middleware.
6. `store(Request $request)` receives the submitted data. `validate()` checks the rules. If it fails, the insert is never reached.
7. `Product::create($data)` inserts a row using the permitted fillable fields.
8. The database assigns its ID, and Eloquent manages its timestamps.
9. The controller redirects to `product.index` with a temporary success message.

Example input: Apple, quantity 12, price 25.00, description Fresh red apples. The ID is generated; do not assume it will be 1 after previous test deletions.

Read: display products

1. `GET /product` calls `index()`.
2. `Product::all()` reads all rows into an Eloquent Collection.
3. `compact('products')` creates an array equivalent to `['products' => $products]`, making `$products` available in the view.
4. `@forelse` displays one row per Product object. `@empty` handles zero records.
5. `number_format($product->price, 2)` displays two decimal places.

`$products` means the collection; `$product` means one record during the loop. Formatting a displayed price does not validate submitted prices.

Update: change quantity to 15

1. The Edit link generates `/product/{id}/edit` using the selected model.
2. The `{product}` placeholder matches the controller's `Product $product`. Laravel's route model binding resolves the record, normally by ID.
3. `edit()` passes that record to `products.edit`.
4. `old('qty', $product->qty)` initially displays the saved quantity. If a validation failure occurred, the previous submitted value takes priority.
5. The edit form submits POST with `@method('PUT')`; Laravel matches the PUT route.
6. `update()` validates, then `$product->update($data)` changes the existing row.
7. The ID and creation time remain; Eloquent updates the modification timestamp.

The word `update` describes changing a saved record, not opening the edit form.

Delete: remove a temporary record

1. Delete is a submit button inside a form for the selected product.
2. `confirm()` asks the user. Cancel returns false and prevents this submission.
3. OK submits POST with the CSRF token and hidden DELETE method value.
4. The route calls `destroy()` with the bound Product object.
5. `$product->delete()` removes the row and the controller redirects to the list.

This is a permanent delete. We did not add Laravel SoftDeletes or a recycle bin. The confirmation is a user-interface convenience, not a permission check.

5. HTTP methods and route names

Method	URL	Route name	Controller	Meaning
GET	`/product`	`product.index`	`index`	Read/display list
GET	`/product/create`	`product.create`	`create`	Display add form
POST	`/product`	`product.store`	`store`	Save new record
GET	`/product/{product}/edit`	`product.edit`	`edit`	Display edit form
PUT	`/product/{product}`	`product.update`	`update`	Save changes
DELETE	`/product/{product}`	`product.destroy`	`destroy`	Delete record

A route name is a reusable label, not the actual URL. `route('product.index')` generates its URL. The list and store share `/product`, but their HTTP methods are different. GET should not be used to delete records just by visiting a link. Native HTML forms support GET and POST; Laravel's `@method` adds a hidden `_method` field for PUT or DELETE. `GET|HEAD` in route:list is normal.

6. Understand the database and validation

Column	Migration definition	Explanation
id	`id()`	Unique auto-incrementing primary key
name	`string('name')`	Short product name, default length 255
qty	`integer('qty')`	Whole-number quantity
price	`decimal('price', 8, 2)`	Eight digits total, two decimal places
description	`text(...)->nullable()`	Optional longer text; SQL NULL permitted
created_at / updated_at	`timestamps()`	Creation and modification times

`decimal(8, 2)` means up to six digits before the decimal and two after it. Our positive maximum is 999999.99. It does not mean eight digits before the decimal. Decimal storage avoids approximate floating-point storage for prices.

`up()` creates the table; `down()` drops it during rollback. Dropping a table also removes its rows. Laravel records completed migrations in its migrations table, which explains why a second `artisan migrate` says nothing to migrate. A new terminal does not erase database history.

Field	Controller rule	Example
name	required, string, max 255	Apple passes; missing name fails
qty	required, integer, min 0	15 passes; -1 or 1.5 fails
price	required, numeric, min 0, max 999999.99, decimal 0 to 2	25 and 25.00 pass; 1.999 fails
description	nullable, string	Empty is allowed; non-null value must be text

The | characters separate validation rules. Name's `max` limits characters; price's `max`, combined with numeric validation, limits the numeric value.

There are three distinct responsibilities:

- Browser attributes (`required`, `min`, `max`, `step`) help the user catch mistakes early.
- Controller validation enforces rules on the server even when browser HTML is changed.
- Database columns determine how accepted data is stored.

`$fillable` is a fourth concept: it restricts which fields can be assigned together. It does not validate those values or authenticate the user.

For your decimal test, the browser first rejected 1.999 with the nearest allowed values 1.99 and 2.00. Temporarily using `step="any"` allowed the request to reach Laravel, which displayed the 0-2 decimal-places error. You restored `step="0.01"`. When validation fails, Blade shows `$errors`, and `old()` restores submitted input.

7. Blade, PHP, CSS, and SVG terms to explain

Code	Meaning here
`namespace`	Organizes a PHP class under a name such as App\Models
`use`	Imports a class name so the file can refer to it conveniently
`extends`	Inherits behavior from a parent class
`public function`	Defines a method the routes can call
`$variable`	Stores a PHP value
`->`	Accesses an object's method or property
`::`	Accesses a class-level member; used in our Eloquent calls and class reference
`=>`	Connects an array key to its value, such as a field to its rules
`return`	Gives a result, view, or redirect back to the caller
`@extends`	Uses a shared Blade layout
`@section` / `@yield`	Provide page content / insert it into the layout
`{{ ... }}`	Display escaped output so entered HTML is treated as text
`@if`	Display a block only when its condition is true
`@foreach`	Repeat a block for every value
`@forelse` / `@empty`	Loop with an alternative when the collection is empty
`@csrf`	Add the hidden request-forgery protection token
`@method`	Supply PUT or DELETE through an HTML POST form
`asset()`	Generate a URL for an asset in public/
`session('success')`	Read the temporary feedback set after a CRUD operation

The application's teaching comments use `//` in PHP and JavaScript, with neutral, beginner-friendly developer explanations of purpose, behavior, and connections. Related lines form one explanatory paragraph without "I" or "In my code" phrasing. Existing PHPDoc `/** ... */` blocks are retained. In Blade, these `//` explanations are inside `@php` / `@endphp` blocks. They are executed as comment-only PHP and produce no visible page text. Bare `//` outside a PHP block would appear on the page. CSS comments use `/* ... */`. They explain code; they are not instructions the browser executes.

CSS classes connect markup to appearance. `.button-primary` selects an element with that class; `.brand .icon` selects an icon inside the brand. Padding is space inside an element; margin is space outside it. Flexbox arranges nearby elements, and wrapping lets controls move to the next line.

The glass appearance combines translucency (`rgba`), background blur (`backdrop-filter`), rounded borders, and a soft shadow. With the mostly neutral background it is intentionally subtle. Browsers without backdrop blur still display the

dark surface. The font stack now uses Inter, loaded through Google Fonts in the shared layout. System fonts are fallbacks if the download fails.

The reference refinement adds layered gradients and inset shadows for soft silver reflections. An inset shadow is drawn inside the panel rather than outside it. The main blue button now has a brighter top and darker bottom. The live table height still depends on your actual number of records, and the success banner only appears after an operation; reference images use example data.

Light mode uses neutral pearl-and-graphite lighting behind clear frosted navigation and a quieter translucent content panel. Reflected edge highlights, blur, and soft shadows approximate glass while preserving readable table text. Blue actions, coral deletion, and green success keep the same meanings in both themes. Layered CSS gradients are listed in front-to-back order.

The sun button uses public/js/theme.js. A click listener calls setTheme(), which adds or removes light-theme on body. CSS rules such as .light-theme .brand then override the relevant colors without changing CRUD. localStorage remembers the choice between pages in this browser. try/catch handles browsers that block storage; the button still works for the current page. aria-label explains the next action to screen readers, and aria-pressed reports the selected light state.

The table keeps a minimum readable width and scrolls inside its container on small screens. The page does not need to squeeze six columns into a tiny width. `@media (max-width: 600px)` adjusts spacing and type. Focus outlines help keyboard users, and reduced-motion rules remove the button transition.

SVG icons are vector drawings. `viewBox="0 0 24 24"` establishes coordinates, `path d` contains line/curve instructions, `fill="none"` makes outlines, `stroke="currentColor"` inherits the text color, and `stroke-width` sets thickness. `aria-hidden` hides decorative icons from screen readers because visible labels already name each action. Blue marks main/edit actions; coral identifies delete; green indicates success. Text labels ensure that color alone is not required.

8. Local setup and demonstration commands

We observed XAMPP PHP 8.2.12, Composer 2.8.12, and Laravel Framework 12.69.3. XAMPP bundles MariaDB 10.4.32, a MySQL-compatible database; Laravel connects through its `mysql` driver. Our configured database is `paraguya_crud_lab2` on 127.0.0.1, port 3307. This port differs from the common default 3306.

Your default terminal PHP points to Herd PHP 8.5. Explicitly using the XAMPP executable keeps the lab commands consistent. The Laravel installer version is different from the framework version inside the application.

Start XAMPP MySQL, then run in PowerShell:

```
cd "C:\Users\Yestin Paraguya\Downloads\Paraguya_CRUD_Lab2\app-crud"
& "C:\xampp\php\php.exe" artisan --version
& "C:\xampp\php\php.exe" artisan route:list --path=product
& "C:\xampp\php\php.exe" artisan migrate:status
& "C:\xampp\php\php.exe" artisan serve
```

Open http://127.0.0.1:8000/product. Keep that terminal running; Ctrl+C stops the development server. It uses PHP's development server, so XAMPP Apache is not needed to serve this URL. Apache may be needed for XAMPP's phpMyAdmin URL. Use the URL/port actually configured on your laptop rather than assuming port 80.

`.env` selects connection, host, port, database, username, and password. It also contains the application key and other settings. Do not display or include credentials/application keys in screenshots. After changing database settings, `artisan config:clear` removes cached configuration. It does not delete records.

9. Practice demonstration and suggested screenshots

Use a disposable product for deletion. Keep your main Apple record.

1. Show the list and explain where its data comes from.

2. Add Demo Notebook, quantity 5, price 45.00, description Demo record.
3. Show the new row and the success message.
4. Edit quantity to 8; show that the same ID remains after updating.
5. Test an invalid price if asked. Restore any temporary input change immediately.
6. Cancel deletion once to show that nothing is removed, then confirm deletion.
7. Show the list after deletion, including the feedback message.
8. Narrow the window and show local table scrolling and the fitted forms.
9. Open the code in this order: routes, controller, model, migration, views, CSS.

Suggested evidence to capture with readable text:

Screenshot	What it proves
Product list with saved rows	Read operation
Filled Add Product form and saved result	Create operation
Edit form and updated result	Update operation
Delete result with success message	Delete operation
Laravel validation alert	Server-side invalid-input handling
Database products structure and rows	Migration schema and persisted data
Narrow table and forms	Responsive behavior
Six product routes	URLs mapped to controller methods

If a database export is required, export this lab database using phpMyAdmin and verify it includes both structure and data. Do not drop/reset the database to prepare a demonstration. Follow the instructor's actual submission format.

10. Practice answers to likely questions

What is CRUD? Create inserts a record, Read retrieves it, Update changes an existing record, and Delete removes it. Point to store/index/update/destroy.

Why are there six methods for four operations? Creating and updating each need a method to display the form and another to process its submission.

What is Eloquent? Laravel's ORM: it maps tables and rows to model classes and objects. Our controller calls Product methods instead of hand-writing SQL.

Why Product and products? Laravel uses a plural table name by convention. Product represents the products table unless a different table is configured.

Does the migration save Apple? No. It creates the table. `store()` and `Product::create()` save Apple as a row inside that table.

Where is validation? In store() and update() in ProductController. The browser attributes help early, but server rules are required because HTML can be changed.

Why use fillable? To declare the fields allowed for mass assignment. It is separate from checking whether a price or quantity is valid.

How does Edit know which row? The URL contains the product's route key, normally its ID. Route model binding supplies that Product to the controller. An ID that is not found normally gives 404.

Why use old()? It restores previous submitted input after an error. Edit's second argument supplies the saved value when there is no previous input.

Why csrf? The session token helps Laravel reject cross-site forged form requests. It does not replace user authentication or access permissions.

Why POST on an update/delete form? HTML forms do not directly support PUT or DELETE. Laravel reads the hidden method field generated by @method.

Why a shared layout? It centralizes the navbar, document shell, and CSS link. Each child view supplies title/content sections, reducing repeated markup.

Why did nothing to migrate appear? Laravel had already recorded those migrations as run. Restarting the terminal does not reset that history.

Why not lucide-react? The assignment uses PHP/Laravel Blade. Inline SVG achieves the desired outline appearance without introducing React.

Is it ready for a public production system? It meets our local CRUD lab scope. It has no login or authorization, pagination, search, or soft-delete recovery. Do not describe it as a full inventory platform or claim features we did not build.

11. A short explanation you can say in your own words

"My Laboratory 2 project is a product CRUD application built with Laravel MVC. The routes connect each request to ProductController. The controller validates submitted data and uses the Product model with Eloquent to work with the products table. The migration defines that table's columns. Blade views display the product list and the create/edit forms, and they share one layout. I added plain CSS and SVG icons for the dark glass design. I tested all four CRUD operations, invalid prices, and the layouts at a narrow screen width."

Practice explaining one concrete request rather than trying to recite every file: start at Add Product, follow its POST route into store(), explain validation and the insert, then follow the redirect back to the list.

12. Technical references

Both themes use Inter, loaded once through Google Fonts in the shared layout. Weights 400, 500, 600, and 700 cover body text and headings; display=swap keeps text visible while downloading. Buttons and inputs inherit the body font. If the hosted font cannot load, system-ui or Segoe UI is used instead. font-optical-sizing: auto enables size-aware outlines when supported by the font. The glass effect uses ordinary CSS rather than Apple's native refraction renderer.

Apple's Liquid Glass introduction describes translucent controls and navigation as a layer above content.

These support the concepts above; your own code and activity material remain the main sources for explaining what you implemented.

- Laravel 12 routing, model binding, and method spoofing
- Laravel 12 Eloquent models and mass assignment
- Laravel 12 validation
- Laravel 12 Blade templates

Activity reference: `C:/Users/Yestin Paraguya/Downloads/Documents/Activity no.2 _ Laravel CRUD.pdf`. Screenshot references: `C:/Users/Yestin Paraguya/Downloads/LAB 2 CCC181/Step2.png` through `Step7.png` supplied in this conversation.

13. Verification after adding teaching comments

On October 7, 2026, the six edited PHP files passed syntax checks. All six product routes were present, the products migration was marked Ran, and the connected database contained id, name, qty, price, description, created_at, and updated_at. Laravel compiled the Blade views and rendered the list, create, and edit views with their shared layout successfully. No product records were inserted, updated, or deleted during this verification.

The application files were compared with their pre-comment versions: PHP tokens excluding comments/whitespace, Blade content excluding Blade comments/whitespace, and CSS excluding comments/whitespace matched. The application logic and styling were preserved. Your earlier browser demonstrations provide the interaction and responsive-layout evidence; this pass checked syntax, rendering, routes, and schema.

14. Quick start without VS Code

VS Code is the editor, not the server. The website can run with VS Code closed.

1. Open XAMPP Control Panel and make sure MySQL is running.
2. Open a separate Windows PowerShell window from the Start menu.
3. Run the commands below, then leave that PowerShell window open.
4. Open http://127.0.0.1:8000/product in your browser.

```
cd "C:\Users\Yestin Paraguya\Downloads\Paraguya_CRUD_Lab2\app-crud"
& "C:\xampp\php\php.exe" artisan serve
```

Ctrl+C or closing the server window stops the website. Closing VS Code does not stop a server running in an independent PowerShell window. Save editor changes before closing VS Code.

Service	Port	What it does
Laravel development server	8000	Serves the website through artisan serve
Apache (proposed alternate port)	8080	Serves XAMPP pages and phpMyAdmin after configuration
XAMPP MySQL/MariaDB	3307	Stores this lab database and its records
FileZilla	Separate FTP service	Not required for the website or database inspection

Apache is not required for the Laravel URL above. Running phpMyAdmin through XAMPP does require Apache. Port 8080 was recommended when port 80 was occupied; its successful configuration has not been demonstrated in this conversation.

15. Open and inspect the database in phpMyAdmin

If Apache has been configured to port 8080 and started, open http://localhost:8080/phpmyadmin/. The phpMyAdmin configuration inspected during this lab points to database port 3307, matching Laravel's database connection.

1. Select paraguya_crud_lab2 from the database list.
2. Select the products table.

3. Choose Browse to see saved rows; choose Structure to see the columns.
4. Add or update a product through the website, then refresh Browse to observe the persisted change. This demonstrates that the list is backed by a database.

If Apache still reports port 80 occupied, open its httpd.conf configuration: change Listen 80 to Listen 8080 and ServerName localhost:80 to ServerName localhost:8080. Set Apache's Main Port to 8080 in XAMPP's Service and Port Settings as well, save, and restart Apache. These are setup instructions, not confirmation that the machine has already been changed. If another port error appears, read the new log rather than stopping unknown Windows services.

16. Troubleshooting and final preparation

Symptom	Meaning / next check
Could not open input file: artisan	The terminal is outside app-crud; change to the project folder
Website connection refused	artisan serve is stopped, or the browser URL does not match its reported port
Database connection refused	Check XAMPP MySQL is running and Laravel DB_PORT is 3307
View products.index not found	The file belongs in resources/views/products/index.blade.php
Nothing to migrate	Already-run migrations are recorded in the database; this is normally expected
CSS changes not visible	Edit public/css/style.css and refresh with Ctrl+F5
Text font looks different	Inter could not download; the browser is using the system fallback
Invalid price rejected	Correct the value to at most two decimal places; do not remove server validation
Apache port 80 occupied	Use the documented alternate Apache port for phpMyAdmin

Before the demonstration, check Create, Read, Update, Delete, validation, the light/dark switch, and narrow-screen forms in the current browser version. Earlier CRUD tests passed; the latest visual adjustments still need a final browser comparison against the chosen reference.

Confirm the required submission format with the instructor. Prepare readable screenshots and, if requested, a SQL database export. Keep application source, documentation, and database evidence together. The tmp/ folder and PDF-generation helpers are preparation tools, not part of the Laravel application runtime. Do not include credentials or the application key in screenshots.

For revision, explain one complete request without reading the guide: identify the route, controller method, validation, model operation, database change, and view/redirect. Then open the corresponding code to show the explanation matches the implementation.